Deepening the statistical trial of the halo effect

The Speed Dating dataset supports far more causal rigor than a basic Elastic Net pipeline — and every extension stays within linear models. By combining participant fixed effects in a linear probability model, group-aware cross-validation, Double/Debiased Machine Learning with Lasso nuisance learners, and Cinelli-Hazlett sensitivity analysis, this project can move from descriptive multicollinearity observation to credible causal identification of the halo effect. All methods below use OLS/GLS/penalized linear regression and are implementable in Python with well-maintained packages. The key insight tying everything together: Frisch-Waugh-Lovell is the pedagogical bridge from your OVB analysis to DML, and participant fixed effects transform a cross-sectional correlation story into a within-person causal design.

---

1. The dataset has 8,378 dyadic observations with a treacherous scale discontinuity

The Fisman-Iyengar Speed Dating dataset contains **8,378 rows** (each row = one 4-minute speed date from one participant's perspective) across **195 columns**, generated by **~552 participants** (277 men, 274 women) at Columbia University across **21 waves** from October 2002 to April 2004. The data is available on Kaggle (`annavictoria/speed-dating-experiment`) and the original Columbia mirror (`stat.columbia.edu/~gelman/arm/examples/speed.dating/`), along with the essential 15-page codebook (`Speed Dating Data Key.doc`).

Nesting structure. Each participant (`iid`) rates ~5–22 partners (`pid`) within their wave. The data is dyadic: for every pair (A, B), two rows exist — A rating B and B rating A. This means observations are clustered at both the participant and partner level, which has direct consequences for standard error computation and cross-validation strategy (detailed below).

Core variable groups for the project:

- **Outcomes:** `dec` (own yes/no decision), `dec_o` (partner's decision), `match` (mutual yes, ~16.5% rate)
- **Partner ratings (1–10 scale, all waves):** `attr`, `sinc`, `intel`, `fun`, `amb`, `shar`, `like`, `prob`
- **Pre-survey stated preferences:** `attr1_1` through `shar1_1` — the 100-point allocation task ("distribute 100 points across 6 traits reflecting what you look for")

- **Partner's stated preferences:** `pf_o_att`, `pf_o_sin`, `pf_o_int`, `pf_o_fun`, `pf_o_amb`, `pf_o_sha`
- **Demographics:** `age`, `race` (6 categories), `field_cd` (18 coded fields), `income` (Census median by ZIP, stored as string with commas — must parse), `mn_sat` (undergraduate institution SAT median), `goal`, `date`, `go_out`
- **Activity interests:** 17 interest variables (`sports` through `yoga`, 1–10 scale), with `int_corr` giving the correlation between participant's and partner's interest profiles

The Wave 6–9 scale problem is the single most important data-cleaning gotcha. In Waves 1–5 and 10–21, the stated preference variables (`attr1_1` etc.) use a **constrained 100-point allocation** — participants distribute exactly 100 points across 6 traits. In **Waves 6–9**, participants instead rated each trait independently on a **1–10 scale** with no sum constraint. These are fundamentally incompatible measures. Most researchers either **exclude Waves 6–9 entirely** or normalize them to percentages. For a project emphasizing methodological rigor, exclusion is cleaner and more defensible — note this explicitly as a robustness choice and then run a leave-these-waves-in sensitivity check with normalization.

Additional data quality issues: ~79 observations have systematic missingness across demographic and activity variables (incomplete pre-surveys); follow-up survey variables have **up to 65% missingness**; the `income` field requires string-to-numeric conversion; `field` is free-text (use `field_cd` instead); some participants' 100-point allocations don't perfectly sum to 100 (renormalize); and 10 rows have missing `pid` (all Wave 5, `partner=7`, recoverable from wave structure). Wave 5 used undergraduates rather than graduate students, and Wave 12 imposed a "budget constraint" (could only say yes to 50% of partners).

---

2. What Fisman et al. actually found — and what remains contested

The benchmark result uses LPM with participant fixed effects

A common misconception is that the original paper used conditional logit. In fact, Fisman et al. (2006, QJE) primarily used a **linear probability model with subject fixed effects and robust standard errors clustered by partner** — exactly the model this project should adopt. They note: "We use a linear probability model for ease of interpretation; similar results are obtained with a conditional logit model." Observations were weighted by the inverse of session size to avoid overweighting large events.

Their core finding: **attractiveness coefficients were 0.140 for men vs. 0.119 for women** (both $p<0.01$), with the interaction Attractiveness×Male = +0.020 ($p<0.01$), meaning men weight attractiveness roughly 18% more. Intelligence mattered roughly twice as much for women (0.045 vs. 0.023). They also found that women become more selective as group size increases while men do not, and that women respond significantly to objective proxies of partner quality — partner's undergraduate SAT scores and ZIP code income — while men show no such response.

The 2008 paper in the *Review of Economic Studies* documented significant racial homophily: women were **14.3 percentage points** more likely to accept a same-race partner, with stronger same-race preferences among Black and Asian participants. Growing up in racially diverse neighborhoods reduced this preference.

The stated-vs-revealed gap is settled; the gender difference is not

Settled findings across multiple replications (Eastwick & Finkel 2008; Kurzban & Weeden 2005; Todd et al. 2007; Hitsch, Hortaçsu & Ariely 2010): attractiveness is the dominant predictor of speed-dating decisions for both genders, stated preferences diverge substantially from revealed preferences (correlations between stated importance and actual predictive weight are $r = .00$ to $.17$), and racial homophily effects are real.

Actively debated: Whether the gender difference in attribute weights is real or artifactual. Eastwick & Finkel (2008, JPSP) ran their own speed-dating experiment at Northwestern and found **no sex differences in revealed preferences** — both genders weighted attractiveness and earning prospects equally. Multiple replications (Selterman et al. 2015) confirmed this null. The discrepancy may stem from different statistical approaches, different samples (Columbia grad students vs. Northwestern undergrads), or the fact that attractiveness is the only trait reliably assessable in 4 minutes. This ambiguity is actually useful for the project narrative: the halo effect (multicollinearity in ratings) makes it genuinely hard to isolate what traits matter independently.

Bhargava & Fisman (2014, *Review of Economics and Statistics*) used the same dataset to document **sequential contrast effects** — a prior partner's attractiveness reduces the likelihood of accepting the next partner, driven almost entirely by male evaluators. This is relevant background but not central to the halo effect project.

---

3. Methodological depth that stays strictly within linear models

3a. Your cross-validation is leaking — GroupKFold is non-negotiable

This is the single most impactful methodological correction. Because each participant appears in **10–22 rows**, standard k-fold CV will split the same person's decisions across train and test sets. The model learns participant-specific patterns (baseline pickiness, rating tendencies) from the training fold and then "predicts" that same participant in the test fold — inflating performance estimates and biasing regularization parameter selection toward underfitting.

The fix: `GroupKFold` on `iid`. Every fold must contain all-or-none of a participant's dates. In scikit-learn, `ElasticNetCV` does not natively pass `groups` to its internal splitter, so you must either pre-generate splits as a list of (train_idx, test_idx) tuples from `GroupKFold(n_splits=5).split(X, y, groups=df['iid'])` and pass that list to `cv=`, or use `GridSearchCV` with `ElasticNet` base estimator and `GroupKFold` as the CV object (since `GridSearchCV.fit()` does accept and forward the `groups` argument). The latter approach is more robust and canonical. Report the CV score difference between naive k-fold and GroupKFold — the gap itself is a finding about information leakage.

3b. Beyond basic Elastic Net: adaptive Lasso and stability selection

Adaptive Lasso (Zou 2006) has the oracle property — it identifies the true model with probability approaching 1 as sample size grows, unlike standard Lasso which can be inconsistent. Implementation: fit Ridge to get initial coefficient estimates, compute weights $w_j = 1/|\hat{\beta}_j^{\text{Ridge}}|^\gamma$ (with $\gamma \in \{0.5, 1, 2\}$), transform features $\tilde{X}_j = X_j / w_j$, fit Lasso on transformed features, then recover original-scale coefficients by dividing by weights. No dedicated pip package exists; manual implementation with scikit-learn is ~15 lines (shown in Section 4 below). Compare the variable selection between standard Elastic Net and Adaptive Lasso — differences reveal sensitivity of the "revealed weights" to the penalty structure.

Stability selection (Meinshausen & Bühlmann 2010) provides a principled answer to "which variables are robustly selected?" by running Lasso on many random subsamples (typically n/2 observations, 100+ iterations) across a grid of regularization strengths and computing selection probabilities. A variable is "stable" if it appears in >60% of subsampled models. The `stability-selection` package (scikit-learn-contrib) implements this. The stability path plot — selection probability vs. regularization strength for each variable — is a powerful visualization showing that `attr` has near-100% selection probability while `amb` might hover at 20%.

Post-Lasso OLS (Belloni & Chernozhukov): Run Lasso to select variables, then refit unrestricted OLS on only the selected variables. This avoids the shrinkage bias of Lasso coefficients while retaining the variable selection benefit. The "revealed weights" computed from Post-Lasso OLS coefficients are more interpretable than from regularized coefficients.

3c. The linear probability model is not just defensible — it's what Fisman et al. used

OLS on a binary outcome (LPM) is the standard in applied economics. Its advantages here: coefficients are marginal effects (a 1-point increase in attractiveness rating increases the probability of "yes" by β percentage points), it accommodates fixed effects trivially, and it permits direct comparison with the original Fisman et al. results. The textbook concerns — predictions outside [0,1] and heteroskedasticity — are handled by using **heteroskedasticity-robust standard errors** (or cluster-robust, which subsumes this). For a course constrained to linear models, LPM is the correct choice. If the course considers GLMs as "linear" (they have a linear predictor), conditional logit would be the alternative, but LPM is strictly more within-scope and more interpretable.

3d. Participant fixed effects transform the analysis from correlation to causation

This is the highest-impact addition to the project. Including participant fixed effects (`iid` dummies, or equivalently the within-transformation) in the LPM means every coefficient is identified **from within-person variation** — how the same person's decision varies across partners who differ in rated attractiveness, sincerity, etc. This controls for all time-invariant participant characteristics: baseline pickiness, mood that evening, personal attractiveness, age, race, field, income, personality, dating goals — everything. You don't need to measure or include these covariates; the fixed effect absorbs them all.

With participant FE, the halo effect coefficient on `attr` answers: "When participant i rates partner A one point more attractive than partner B, holding the sincerity, intelligence, fun, ambition, and shared-interests ratings constant, how much more likely is participant i to say yes to partner A?" This is a far stronger causal claim than the pooled cross-sectional estimate.

Implementation uses the `linearmodels` package: set the DataFrame index to `(iid, order)` (where `order` is the within-session date sequence), then `PanelOLS(y, X, entity_effects=True).fit(cov_type='clustered', cluster_entity=True)`. You can also add **partner fixed effects** for a two-way FE model (à la AKM/Abowd-Kramarz-Margolis) that additionally controls for all time-invariant partner characteristics — partner's "true" attractiveness, charisma, etc. — leaving identification from the interaction-specific residual. Two-way FE is ambitious but doable and would be genuinely impressive in a course project.

Critical: cluster standard errors at the participant level (and ideally two-way cluster at both `iid` and `pid` using Cameron-Gelbach-Miller). In `linearmodels`, `cluster_entity=True` handles one-way clustering; for two-way, you would need `statsmodels` with manual implementation or the `linearmodels` formula interface with custom cluster specifications.

3e. From OVB formula to DML: Frisch-Waugh-Lovell is the bridge

The project's existing OVB analysis (short regression of `dec` on `sinc` alone vs. long regression adding `attr`, applying $\hat{\alpha}_1 = \hat{\beta}_1 + \hat{\beta}_2\hat{\gamma}$) is a perfect pedagogical setup for the Frisch-Waugh-Lovell (FWL) theorem, which in turn motivates Double/Debiased Machine Learning.

FWL says: The coefficient on `attr` in a multivariate regression of `dec` on `attr + sinc + controls` equals the coefficient from regressing the residuals of `dec` (after partialling out `sinc + controls`) on the residuals of `attr` (after partialling out `sinc + controls`). This is exactly what DML does — except DML uses Lasso/Ridge rather than OLS to partial out the controls, gaining robustness to high dimensionality and model selection.

Double/Debiased ML (Chernozhukov et al. 2018) with Lasso nuisance learners is entirely within the linear models constraint — the nuisance functions are estimated with Lasso (a linear model), and the final causal parameter is an OLS coefficient. The `DoubleML` Python package makes this straightforward: specify `dec` as outcome, `attr` as treatment, and `[sinc, intel, fun, amb, shar, age, race, income, ...]` as controls. Lasso handles the high-dimensional nuisance estimation; cross-fitting eliminates overfitting bias. The output is a **debiased causal estimate of the effect of attractiveness on the decision**, with valid confidence intervals. This is genuinely cutting-edge applied econometrics, published in 2018 and rapidly becoming standard.

3f. Sensitivity analysis gives the project a credible causal capstone

Cinelli & Hazlett (2020, JRSS-B) "Making Sense of Sensitivity" extends OVB analysis into a formal sensitivity framework. The key quantities: the **Robustness Value (RV)** tells you how strong an omitted confounder would need to be (in terms of partial R^2 with both treatment and outcome) to explain away the entire estimated effect. If RV = 0.15, an omitted confounder would need to explain at least 15% of the residual variance in both `attr` and `dec` to nullify the halo effect — and you can benchmark this against observed covariates (e.g., "a confounder 3× as strong as gender would still not overturn the result"). The `PySensemakr` package produces publication-quality contour plots. This is ideal for the project because it directly generalizes the OVB formula they already understand.

Oster (2019) bounds offer a complementary approach: assuming that selection on observables is proportional to selection on unobservables ($\delta = 1$) and specifying an R-max (typically $1.3 \times R^2_{\text{full}}$ or $R_{\max} = 1$), compute the bias-adjusted coefficient $\beta^* = \hat{\beta}_{\text{full}} - \delta \cdot \frac{(\hat{\beta}_{\text{short}} - \hat{\beta}_{\text{full}})(R_{\max} - R^2_{\text{full}})}{R^2_{\text{full}} - R^2_{\text{short}}}$. If $\beta^* > 0$, the result is robust to proportional selection. **No pip-installable Python package exists** — only a Stata command (`psacalc2`) and an R package (`robomit`). However, the formula is simple enough for a 10-line Python function (provided in Section 4). I'd recommend PySensemakr (Cinelli-Hazlett) as the primary sensitivity tool and Oster as a secondary robustness check.

3g. Heterogeneity analysis with interaction terms

Does the halo effect differ by gender? By race? By whether the participant claimed to value looks? These are testable with interaction terms in the LPM — all strictly linear. The most interesting test: interact `attr` with `attr1_1` (stated importance of attractiveness). If the halo effect is stronger for people who claim not to care about looks, that's a striking finding about self-deception. Gender interactions replicate/challenge Fisman et al.'s core finding and connect to the Eastwick & Finkel debate. Race interactions connect to Fisman et al. (2008). All are single-line additions to the regression specification.

---

4. Python implementation guide with concrete code patterns

GroupKFold with Elastic Net (the correct way)

```
from sklearn.linear_model import ElasticNet
from sklearn.model_selection import GridSearchCV, GroupKFold
import numpy as np

param_grid = {
    'alpha': np.logspace(-4, 1, 50),
    'l1_ratio': [0.1, 0.5, 0.7, 0.9, 0.95, 0.99, 1.0]
}
gkf = GroupKFold(n_splits=5)
grid = GridSearchCV(
    ElasticNet(max_iter=10000), param_grid,
    cv=gkf, scoring='neg_mean_squared_error', n_jobs=-1
)
grid.fit(X, y, groups=df['iid'].values)
```

Participant fixed effects LPM with clustered SEs

```
from linearmodels.panel import PanelOLS
import statsmodels.api as sm

df_panel = df.set_index(['iid', 'order'])  # (entity, time) MultiIndex
exog = df_panel[['attr', 'sinc', 'intel', 'fun', 'amb', 'shar']]
mod = PanelOLS(df_panel['dec'], exog, entity_effects=True)
res = mod.fit(cov_type='clustered', cluster_entity=True)
print(res.summary)
```

OLS with cluster-robust SEs (for pooled regressions)

```
import statsmodels.formula.api as smf

model = smf.ols('dec ~ attr + sinc + intel + fun + amb + shar', data=df)
results = model.fit(cov_type='cluster', cov_kwds={'groups':
df['iid'].values})
```

Double ML with Lasso nuisance learners

```
from doubleml import DoubleMLData, DoubleMLPLR
from sklearn.linear_model import LassoCV

controls = ['sinc', 'intel', 'fun', 'amb', 'shar', 'age', 'samerace']
dml_data = DoubleMLData(df, y_col='dec', d_cols='attr', x_cols=controls)
dml_plr = DoubleMLPLR(dml_data, ml_l=LassoCV(), ml_m=LassoCV(),
                      n_folds=5, score='partialling out')
dml_plr.fit()
print(f"Causal effect of attr: {dml_plr.coef[0]:.4f} "
      f"(SE: {dml_plr.se[0]:.4f}, p={dml_plr.pval[0]:.4f})")
```

Cinelli-Hazlett sensitivity analysis

```
import sensemakr as smkr
import statsmodels.formula.api as smf

fitted = smf.ols('dec ~ attr + sinc + intel + fun + amb + shar',
                 data=df).fit()
sens = smkr.Sensemakr(model=fitted, treatment='attr',
                      benchmark_covariates=['sinc', 'intel'],
                      kd=[1, 2, 3], q=1.0, alpha=0.05)
sens.summary()
sens.plot(sensitivity_of='estimate')  # contour plot
```

Oster bounds (manual implementation)

```
import statsmodels.api as sm
import numpy as np

def oster_beta_star(y, X_short, X_full, treat_idx, Rmax, delta=1.0):
    r_short = sm.OLS(y, sm.add_constant(X_short)).fit()
    r_full = sm.OLS(y, sm.add_constant(X_full)).fit()
    b_s, b_f = r_short.params[1], r_full.params[treat_idx + 1]
    R_s, R_f = r_short.rsquared, r_full.rsquared
    return b_f - delta * (b_s - b_f) * (Rmax - R_f) / (R_f - R_s)

# Usage: Rmax = 1.3 * R²_full is Oster's recommended bound
```

Adaptive Lasso (manual)

```
from sklearn.linear_model import Ridge, LassoCV
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_sc = scaler.fit_transform(X)
ridge_coefs = Ridge(alpha=1.0).fit(X_sc, y).coef_
weights = 1.0 / (np.abs(ridge_coefs) ** 1.0 + 1e-10)
X_weighted = X_sc / weights
lasso = LassoCV(cv=list(gkf.split(X_sc, y, groups=groups))).fit(X_weighted,
y)
adaptive_coefs = lasso.coef_ / weights  # recover original-scale coefficients
```

Stability selection

```
from stability_selection import StabilitySelection
from sklearn.linear_model import Lasso
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

selector = StabilitySelection(
    base_estimator=Pipeline([('scaler', StandardScaler()),
                             ('lasso', Lasso())]),
    lambda_name='lasso__alpha',
    lambda_grid=np.logspace(-5, -1, 50),
    threshold=0.6, n_bootstrap_iterations=100
)
selector.fit(X, y)
```

```
# Plot: stability_selection.plot_stability_path(selector)
```

Package summary

Package	Install	Purpose	Status
`scikit-learn`	Built-in	ElasticNet, GroupKFold, GridSearchCV	✅ Active
`statsmodels`	Built-in	OLS, cluster-robust SEs, formula API	✅ Active
`linearmodels`	`pip install linearmodels`	PanelOLS, entity FE, two-way FE	✅ v7.0
`DoubleML`	`pip install DoubleML`	DML with Lasso nuisance	✅ v0.11
`PySensemakr`	`pip install PySensemakr`	Cinelli-Hazlett sensitivity	✅ Available
`stability-selection`	`pip install stability-selection`	Meinshausen-Bühlmann	⚠️ May need sklearn compat fix
`selectinf`	`pip install selectinf`	Post-selection inference	⚠️ Academic

---

5. Recommended project structure: five acts of the halo effect trial

Here is a concrete, feasible project outline that preserves the narrative, adds real methodological depth, and stays within linear models.

Act I — "The witnesses speak": EDA and the stated-revealed gap

Clean the data (drop or normalize Waves 6–9 with explicit justification). Visualize the distribution of 100-point allocations by gender (radar charts or stacked bars). Compute correlation matrices of the six partner rating variables and visualize multicollinearity with a heatmap — the halo effect is visible in the off-diagonal entries (attr-sinc correlation, attr-fun

correlation). Show the stated importance of attractiveness (from `attr1_1`) vs. what actually predicts `dec` in a simple pooled regression. Introduce the "cognitive collinearity" concept: when you find someone attractive, you also perceive them as more sincere, intelligent, and fun, inflating all ratings simultaneously.

Act II — "The interrogation": regularized revealed preferences with proper CV

Run Elastic Net on `[attr, sinc, intel, fun, amb, shar]` predicting `dec`, but with **GroupKFold on `iid`** (not naive k-fold). Report the difference in selected alpha/l1_ratio and CV scores between naive and grouped CV — this methodological comparison is itself a finding. Normalize non-zero coefficients to sum to 100 as "revealed weights." Then run Adaptive Lasso and compare variable selection. Run stability selection and display the stability path — `attr` will dominate at essentially any regularization strength. Run Post-Lasso OLS on the stability-selected variables for unbiased coefficient estimates. Present the revealed weights from multiple methods side-by-side against stated weights. Compute bootstrap confidence intervals (resample participants, not rows, to respect clustering) for the revealed weights.

Act III — "Controlling the courtroom": fixed effects and within-person identification

Introduce participant fixed effects using `PanelOLS`. Explain that with FE, every coefficient is identified from within-person variation, controlling for all time-invariant characteristics. Run the LPM with entity effects and cluster-robust SEs. Compare pooled OLS coefficients to FE coefficients — the change (likely modest for partner ratings, larger for demographic controls which get absorbed) demonstrates what between-person variation was confounding. This is the project's strongest causal identification strategy. Optionally add partner fixed effects (two-way FE) and discuss what additional variation this controls for. Run the heterogeneity analysis: interact `attr` with `gender`, with `attr1_1` (stated attractiveness importance), and with `samerace`. Report with a coefficient plot.

Act IV — "The appeal": OVB, sensitivity analysis, and DML

Start with the existing OVB decomposition ($\hat{\alpha}_1 = \hat{\beta}_1 + \hat{\beta}_2\hat{\gamma}$) for sincerity with and without controlling for attractiveness. Then bridge to FWL theorem: show that the long-regression coefficient on `attr` equals the coefficient from regressing residualized `dec` on residualized `attr` (after partialling out all controls via OLS). Then make the leap to DML: replace OLS partialling-out with Lasso partialling-out, gaining robustness to high-dimensional controls. Report the DML estimate of attractiveness's causal effect alongside the OLS and FE estimates. Run PySensemakr: report the Robustness Value and produce the contour plot.

Benchmark against observed covariates — "an omitted confounder would need to be Kx stronger than gender to explain away the halo effect." Run Oster bounds with $R_{\max} = 1.3 \times R^2_{\text{full}}$ as a secondary check. This act demonstrates that the halo effect is robust to a wide range of causal threats.

Act V — "The verdict": synthesis and robustness

Synthesize findings across methods into a single table comparing coefficients, standard errors, and identification strategies. Run subsample stability checks: men vs. women, excluding Wave 5 undergrads, excluding Wave 12 budget-constraint sessions. Run leave-one-wave-out CV. Produce a "robustness dashboard" figure showing that the attractiveness dominance result holds across every specification. Conclude with the substantive insight: people cannot accurately introspect about what drives their romantic decisions, the halo effect creates systematic multicollinearity that makes naive regression misleading, but within-person identification and sensitivity analysis confirm that attractiveness genuinely dominates speed-dating decisions — it's not just a statistical artifact of correlated ratings.

What makes this project stand out at the graduate level

Three elements elevate this from a standard course project to something approaching publication quality. First, **GroupKFold** — most analyses of this dataset on Kaggle and in teaching contexts ignore the clustered structure entirely, making this a genuine methodological contribution relative to the pedagogical baseline. Second, **participant fixed effects** replicate the exact identification strategy of the original Fisman et al. (2006) paper, grounding the analysis in the published literature rather than ad-hoc machine learning. Third, **DML with Lasso nuisance learners** is a method published in *Econometrica* in 2018 that most graduate students in statistics have not yet encountered — it demonstrates cutting-edge causal inference while staying strictly within the linear models family. The Cinelli-Hazlett sensitivity analysis with its contour plots provides a visually compelling and methodologically rigorous capstone that goes beyond what most applied papers include. Together, these form a coherent narrative arc from descriptive patterns through regularized variable selection to credible causal identification — all within linear models, all implementable in Python with existing packages, and all feasible within a course project timeline.